ST-LINK v2 Debugger & Programming Guide

1. Overview of ST-LINK v2

The **ST-LINK v2** is an in-circuit debugger and programmer for the STM8 and STM32 microcontroller families. It interfaces with the host development environment (STM32CubeIDE, OpenOCD, Keil MDK) over USB and communicates with the target STM32 chip via the **Serial Wire Debug (SWD)** protocol.

---

2. SWD (Serial Wire Debug) Interface vs JTAG

Parameter	SWD (Serial Wire Debug)	JTAG (IEEE 1149.1)
Number of Signal Pins	**2 pins** (`SWDIO`, `SWCLK`)	**4 to 5 pins** (`TDI`, `TDO`, `TCK`, `TMS`, `nTRST`)
Pins Freed for Application	Saves 2–3 pins for GPIO/Peripherals	Consumes more MCU pins
Debug Features	Real-time memory inspection, hardware breakpoints, single-stepping	Identical core debug capabilities + boundary scan
ARM Standard	ARM CoreSight proprietary standard	General IEEE boundary scan standard

---

3. Physical Wiring Connection to STM32 Blue Pill

ST-LINK Pin	Target Blue Pill Pin	Purpose
SWDIO	`PA13` (SWDIO)	Bi-directional data line for commands & register access
SWCLK	`PA14` (SWCLK)	Clock line generated by ST-LINK (typically 1–4 MHz)
GND	`GND`	Common ground reference (Mandatory)
3.3V	`3.3V`	Target power rail supply from ST-LINK

> *[!CAUTION]*
>
> ***Do not connect both 5V and 3.3V*** *simultaneously to the Blue Pill board. Connect only* `3.3V` *from ST-LINK to the Blue Pill's* `3.3V` *pin, or power the Blue Pill via its Micro-USB port and connect only* `SWDIO`*,* `SWCLK`*, and* `GND`*.*

---

4. Boot Jumpers (BOOT0 and BOOT1)

The Blue Pill board has two yellow jumpers labeled `BOOT0` and `BOOT1`:

BOOT0	BOOT1	Boot Mode	Description
0 (Default)	**X**	**Main Flash Memory**	Standard execution. Runs your compiled code from address `0x08000000`.

1	**0**	**System Memory**	Boots internal ROM bootloader (Used for UART flashing via **`PA9`**/**`PA10`**).
1	**1**	**Embedded SRAM**	Code executes from RAM (Address **`0x20000000`**) for development/debugging.

- **Normal Operation with ST-LINK:** Keep **BOOT0 = 0** and **BOOT1 = 0**.

---

5. Debugging Capabilities in STM32CubeIDE

When flashing through the provided [**`Oled_display.launch`**](file:///Users/harinarayanans/Documents/Oled_display/Oled_display.launch) profile:

1. **Flashing**: Erases relevant sectors in Flash memory (**`0x08000000`**) and writes the binary **`.elf`**.

2. **Breakpoints**: Uses ARM Cortex-M3 hardware breakpoint registers (up to 6 hardware breakpoints and 4 watchpoints).

3. **Live Expressions**: Monitors variables like **`currentRtc`**, **`currentState`**, and **`dose1`** in real time without stopping CPU execution.

4. **Fault Tracing**: If code hits a **`HardFault_Handler`**, the debugger reveals the Program Counter (PC), Link Register (LR), and stack trace to pinpoint null-pointer dereferences or bus faults.

---

6. Potential Evaluator Questions & Answers

- **Q: Why is SWD preferred over standard UART bootloader programming?**
- *A: UART bootloader only flashes memory, requiring jumper toggles (**`BOOT0=1`** $\rightarrow$ flash $\rightarrow$ **`BOOT0=0`** $\rightarrow$ reset). SWD provides non-intrusive flashing, source-level stepping, variable inspection, register viewing, and live execution control with zero jumper changes.*
- **Q: What is the reset mode used during debugging?**
- *A: ST-LINK supports "Software System Reset", "Core Reset", or "Hardware Reset via NRST". In Blue Pill boards without the NRST pin connected to the ST-LINK header, "Software System Reset" (**`AIRCR.SYSRESETREQ`**) is typically selected.*
- **Q: What should you do if the debugger returns "Cannot connect to target"?**
- *A: Common causes: 1) Missing common ground; 2) Microcontroller entered deep sleep (**`STOP`**/**`STANDBY`** mode); 3) GPIO remapping disabled SWD pins (**`PA13`**/**`PA14`**). Remedy: Hold the onboard Reset button on Blue Pill, start debug session in "Connect Under Reset" mode, and release Reset when the debugger connects.*